

Relatório Comparativo de Código

Questões 1, 2, 3 e 4 — código original × código proposto

Este relatório reúne as análises realizadas a partir dos quatro arquivos enviados. O objetivo é destacar as principais diferenças entre as implementações originais e as versões propostas, com foco em correção, clareza, robustez, organização e complexidade.

Observação: algumas questões foram analisadas somente a partir do código, sem o enunciado original. Nesses casos, o relatório preserva a interpretação indicada pelo código e sinaliza quando uma decisão depende do enunciado.

Questão	Principal problema / oportunidade	Mudança principal
1	Entrada e clareza na extração de bits	Validação da entrada e simplificação da extração do bit B
2	Variável soma desnecessária e escopo	Cálculo direto da média e variáveis no escopo necessário
3	x_min/x_max não inicializados; divisão inteira	Inicialização correta, double e proteção contra divisão por zero
4	Compensação artificial com soma=1 e contador=-1	Início em zero e exclusão explícita do -1 da soma/contagem

Questão 1 — Manipulação de bits

O programa recebe um valor de 0 a 255 e extrai os bits 2, 1 e 0 para R, G e B, respectivamente. O código original usa deslocamento à direita e máscara 1.

Trecho representativo do código original

```
unsigned char R = (M >> 2) & 1;
unsigned char G = (M >> 1) & 1;
unsigned char B = (M >> 0) & 1;
```

Trecho representativo da versão proposta

```
unsigned int R = (M >> 2) & 1;
unsigned int G = (M >> 1) & 1;
unsigned int B = M & 1;
```

Principais diferenças

- **Correção:** a lógica de extração dos três bits já é adequada no código original.
- **Clareza:** no código proposto, `M & 1` expressa diretamente a extração do bit menos significativo, sem um deslocamento por zero.
- **Robustez:** a versão proposta verifica se a leitura foi válida e se o valor está dentro de 0 a 255.
- **Complexidade:** ambas as versões são $O(1)$ em tempo e $O(1)$ em memória; portanto, não há ganho algorítmico relevante de desempenho.

Tabela comparativa

Aspecto	Original	Proposto
Entrada	unsigned char + scanf	unsigned int + validação
Bit B	<code>(M >> 0) & 1</code>	<code>M & 1</code>
Validação	Não há	Verifica leitura e limite
Complexidade	$O(1)$ / $O(1)$	$O(1)$ / $O(1)$

Questão 2 — Média de três valores

O código lê três valores por iteração, soma os três e calcula a média usando divisão por 3.0. A análise considerou a estrutura presente no arquivo, pois o enunciado não foi fornecido.

Trecho representativo do código original

```
int soma = 0;
int xi_1, xi, xi1;

for(int i = 2; i <= n; i++) {
    scanf("%d %d %d", &xi_1, &xi, &xi1);
    soma = xi_1 + xi + xi1;
    float y = soma / 3.0;
}
```

Trecho representativo da versão proposta

```
const int n = 10;

for (int i = 2; i <= n; i++) {
    int xi_1, xi, xi1;
    scanf("%d %d %d", &xi_1, &xi, &xi1);
    float y = (xi_1 + xi + xi1) / 3.0f;
}
```

Principais diferenças

- **Variável desnecessária:** soma é sobrescrita a cada iteração e não acumula valores. O cálculo pode ser feito diretamente.
- **Escopo:** xi_1, xi e xi1 só são necessários dentro do laço, portanto podem ser declarados ali.
- **Constante:** n não é alterado e pode ser declarado como const.
- **Tipo do literal:** 3.0f é coerente com o resultado float, enquanto 3.0 é double.
- **Complexidade:** ambas as soluções permanecem $O(n)$ em tempo e $O(1)$ em memória.
- **Ressalva:** se a questão for especificamente uma média móvel, o enunciado pode permitir reaproveitar valores entre iterações; isso não foi alterado sem o enunciado.

Tabela comparativa

Aspecto	Original	Proposto
soma	Variável intermediária	Removida
Escopo	Variáveis fora do for	Variáveis dentro do for
n	int	const int
Divisão	3.0	3.0f
Complexidade	$O(n)$ / $O(1)$	$O(n)$ / $O(1)$

Questão 3 — Normalização

O código lê X, depois cinco intensidades, procura mínimo e máximo e calcula $X' = (X - X_{\min}) / (X_{\max} - X_{\min})$.

Trecho representativo do código original

```
unsigned char x, intensidade, x_min, x_max;

for (int i = 1; i <= n; i++) {
    scanf("%hhu", &intensidade);

    if (intensidade < x_min) x_min = intensidade;
    if (intensidade > x_max) x_max = intensidade;
}

unsigned char x_normalizado =
    (x - x_min) / (x_max - x_min);
```

Trecho representativo da versão proposta

```
scanf("%hhu", &intensidade);
x_min = intensidade;
x_max = intensidade;

for (int i = 2; i <= n; i++) {
    scanf("%hhu", &intensidade);

    if (intensidade < x_min) x_min = intensidade;
    if (intensidade > x_max) x_max = intensidade;
}

if (x_max == x_min) {
    printf("Não é possível normalizar: máximo e mínimo são iguais.\n");
    return 1;
}

double x_normalizado =
    (double)(x - x_min) / (x_max - x_min);
```

Principais diferenças

- **Problema crítico:** x_min e x_max são comparados antes de receberem valores, causando comportamento indefinido.
- **Inicialização:** a versão proposta usa o primeiro valor como mínimo e máximo iniciais e só depois compara os demais.
- **Precisão:** armazenar a normalização em unsigned char não permite representar resultados fracionários como 0,33. A versão proposta usa double.
- **Divisão por zero:** se máximo e mínimo forem iguais, o denominador é zero. A versão proposta trata essa situação.
- **Complexidade:** ambas são O(n) em tempo e O(1) em memória.

Tabela comparativa

Aspecto	Original	Proposto
x_min/x_max	Não inicializados	Inicializados com primeiro valor
Normalização	unsigned char	double
Divisão por zero	Não trata	Trata x_max == x_min
Laço	1 a 5	1º valor inicializa; 2º a 5º compara
Complexidade	O(n) / O(1)	O(n) / O(1)

Questão 4 — Soma e quantidade com sentinela

O programa lê números até que o usuário informe -1. O valor -1 funciona como sentinela para encerrar a leitura.

Trecho representativo do código original

```
int numero, soma = 1, contador = -1;

do {
    scanf("%d", &numero);
    soma += numero;
    contador++;
} while (numero != -1);
```

Trecho representativo da versão proposta

```
int numero;
int soma = 0;
int contador = 0;

do {
    scanf("%d", &numero);

    if (numero != -1) {
        soma += numero;
        contador++;
    }
} while (numero != -1);
```

Principais diferenças

- **Problema de clareza:** soma = 1 e contador = -1 funcionam como compensações artificiais pelo processamento do -1.
- **Sentinela:** na versão proposta, -1 não é tratado como dado; ele apenas encerra o laço.
- **Inicialização:** soma e contador começam em zero, representando diretamente o estado inicial.
- **Estrutura:** o do...while é mantido, pois uma leitura precisa ocorrer antes da verificação do sentinela.
- **Complexidade:** ambas são $O(n)$ em tempo e $O(1)$ em memória.

Tabela comparativa

Aspecto	Original	Proposto
soma inicial	1	0
contador inicial	-1	0
-1	É somado e contado, depois compensado	Somente encerra
Clareza	Depende de compensação	Regra explícita
Complexidade	$O(n)$ / $O(1)$	$O(n)$ / $O(1)$

Conclusão geral

As versões propostas não procuram substituir os algoritmos por soluções de complexidade menor quando isso não é possível ou necessário. Nas quatro questões, o foco principal foi tornar o código mais explícito, correto e fácil de manter.

Questão 1: a lógica original de bits já é adequada; as melhorias são principalmente de validação e clareza.

Questão 2: a solução pode ser simplificada removendo uma variável intermediária e restringindo o escopo das variáveis.

Questão 3: é a questão com os problemas técnicos mais importantes: variáveis não inicializadas, perda de precisão e possível divisão por zero.

Questão 4: a principal melhoria é substituir a estratégia de compensação por uma lógica direta, na qual o sentinela não participa da soma nem da contagem.

Em termos de eficiência assintótica, as versões propostas mantêm as mesmas ordens de complexidade dos algoritmos originais: $O(1)$ para a Questão 1 e $O(n)$ para as Questões 2, 3 e 4, com $O(1)$ de memória adicional. Portanto, as diferenças mais relevantes são de correção, robustez, legibilidade e organização, e não de desempenho bruto.